

Pruebas de Software



Pruebas de Software

¿Qué son las pruebas de software?

Las pruebas de software son el proceso de evaluar y verificar que un producto o aplicación de software funciona correctamente, de forma segura y eficiente, de acuerdo con sus requisitos específicos.

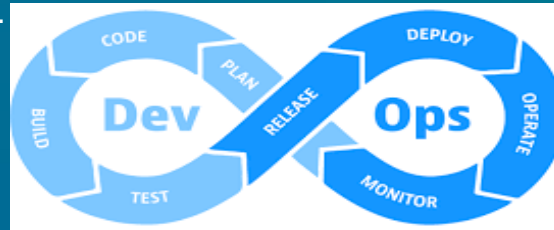
Los principales beneficios de unas pruebas sólidas incluyen la entrega de software de alta calidad mediante la identificación de errores y la mejora del rendimiento.

Hoy en día, las pruebas de software están profundamente integradas en las prácticas de desarrollo modernas, impulsadas por la transformación ágil, DevOps y los pipelines de integración continua/entrega continua (CI/CD).

Pruebas de Software

¿Qué son las pruebas de software?

DevOps es una práctica que une a los equipos de desarrollo (Dev) y operaciones (Ops) de software para crear y entregar aplicaciones y servicios de forma más rápida, fiable y constante. Implica una combinación de cultura, prácticas y herramientas que automatizan el ciclo de vida del desarrollo, desde la planificación y el diseño hasta la entrega y el monitoreo, con el objetivo de mejorar la colaboración, la eficiencia y la calidad del producto.



Un pipeline de Integración Continua/Entrega Continua (CI/CD) es un flujo de trabajo automatizado que optimiza el ciclo de vida del desarrollo de software, desde la escritura del código hasta su despliegue en producción. La Integración Continua (CI) automatiza la compilación y las pruebas del código cada vez que los desarrolladores lo integran en un repositorio centralizado, detectando errores tempranamente. La Entrega Continua (CD) amplía esto, automatizando el proceso de llevar esos cambios probados a un entorno de producción o preparación para ello. Este proceso aumenta la calidad, la velocidad y la fiabilidad de las entregas de software al reducir la intervención manual y los riesgos asociados.

Pruebas de Software

¿Qué son las pruebas de software?

Las pruebas ya no son un paso final antes del lanzamiento, sino que comienzan en la fase de planificación del diseño y continúan después de la implementación.

Este enfoque de pruebas permite lanzamientos más rápidos y reduce el riesgo en entornos de infraestructura de TI que cambian rápidamente. Prácticas como las pruebas shift-left, en las que las pruebas comienzan antes en el ciclo de desarrollo, ayudan a los equipos a detectar problemas antes. Las pruebas shift-right, centradas en la monitorización y la validación en producción, permiten a los equipos adaptarse con mayor rapidez al uso en el mundo real.



Pruebas de Software

¿Qué son las pruebas de software?

Las pruebas shift-left (desplazamiento a la izquierda) son un enfoque de aseguramiento de la calidad del software que consiste en integrar y realizar las actividades de prueba en las etapas más tempranas del ciclo de vida del desarrollo de software, en lugar de esperar hasta el final. Esta práctica tiene como objetivo detectar y corregir errores lo antes posible, lo que resulta en una solución más rápida, económica y de mayor calidad, ya que los defectos son más fáciles y baratos de solucionar cuando se encuentran temprano.

Las pruebas Shift-Right (desplazamiento a la derecha) implican realizar pruebas y monitoreo en el entorno de producción o cercano a este, después del lanzamiento de un software, para recopilar datos de usuarios reales y asegurar su rendimiento en un entorno real. A diferencia del Shift-Left, que detecta problemas temprano, el Shift-Right identifica escenarios inesperados y errores que solo ocurren en producción, utilizando técnicas como monitoreo, pruebas en producción controlada (smoke tests, canary releases) y la recopilación de retroalimentación del usuario.

Pruebas de Software

¿Qué son las pruebas de software?

Las pruebas en producción controlada, que incluyen los smoke tests (pruebas de humo) y las canary releases (lanzamientos canario), son estrategias de validación en entornos reales para minimizar riesgos antes de una implementación completa. Los smoke tests verifican rápidamente la funcionalidad básica y la estabilidad de una nueva compilación de software. Por otro lado, las canary releases lanzan una nueva versión de forma gradual a un pequeño subconjunto de usuarios o servidores ("canarios"), permitiendo detectar y solucionar problemas antes de que afecten a todos, usando la analogía de los canarios en las minas de carbón para detectar gases peligrosos.

Las estrategias modernas de pruebas de software siguen evolucionando a la par de los avances en automatización, inteligencia artificial (IA) y arquitecturas nativas de la nube, como los microservicios. A medida que el software se vuelve más complejo y los ciclos de lanzamiento se aceleran, las pruebas inteligentes se han vuelto cada vez más frecuentes.

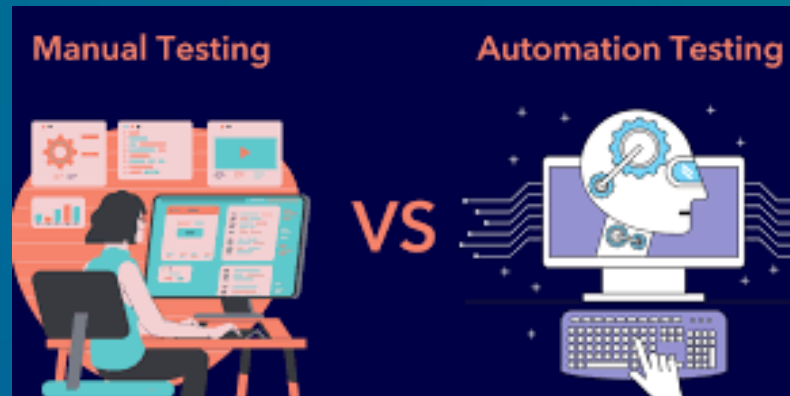
Pruebas de Software

¿Por qué son importantes las pruebas de software?

Cuando el desarrollo deja un amplio margen para las pruebas, mejora la fiabilidad del software y se obtienen aplicaciones de alta calidad con menos errores. Un sistema que cumple o supera las expectativas de los clientes puede traducirse en un aumento de las ventas, una mayor cuota de mercado y una mejor experiencia para los usuarios.

Las pruebas de software se dividen principalmente en dos grandes categorías:

- Pruebas manuales
- Pruebas automatizadas



Pruebas de Software

¿Por qué son importantes las pruebas de software?

Pruebas manuales

Las pruebas manuales son el proceso mediante el cual los evaluadores ejecutan casos de prueba manualmente sin la ayuda de herramientas de automatización. Los evaluadores realizan acciones como hacer clic en botones, introducir texto y verificar los resultados, simulando cómo interactuaría un usuario final con el software.

Las pruebas manuales se suelen utilizar para pruebas exploratorias, pruebas de usabilidad y cuando la aplicación es lo suficientemente pequeña como para que no sea necesario automatizarla.

Pruebas automatizadas

Las pruebas automatizadas utilizan scripts y herramientas para ejecutar pruebas en el software de forma automática. Este enfoque fundamental es beneficioso para las tareas de prueba repetitivas y para los sistemas más grandes en los que es necesario ejecutar las mismas pruebas varias veces.

Las pruebas automatizadas garantizan que el software se pueda probar de forma más rápida y coherente. También reducen el error humano y mejoran la eficiencia de las pruebas con el tiempo.

Pruebas de Software

Niveles de pruebas de software

Las pruebas de software se producen en cuatro niveles diferentes dentro del ciclo de vida del desarrollo del software, cada uno de los cuales se centra en partes específicas de la aplicación:

- Pruebas de pruebas de unidad
- Pruebas de integración
- Pruebas del sistema
- Pruebas de aceptación



Pruebas de Software

Niveles de pruebas de software

Pruebas de pruebas de unidad: Las pruebas unitarias validan que cada unidad de software funcione según lo esperado. Una unidad es el componente comprobable más pequeño de una aplicación.

Pruebas de integración: Las pruebas de integración garantizan que los componentes o funciones de software funcionen juntos de manera efectiva.

Pruebas del sistema: Las pruebas del sistema implican el rendimiento integral de todo un sistema. Esta fase incluye aspectos de pruebas funcionales, pruebas no funcionales, pruebas de interfaz, pruebas de estrés y pruebas de recuperación.

Pruebas de aceptación: Las pruebas de aceptación verifican si todo el sistema funciona según lo previsto.

Pruebas de Software

Tipos de pruebas de software

Existen muchos tipos diferentes de pruebas de software que se engloban en los niveles mencionados anteriormente, y normalmente se pueden dividir en dos categorías principales:

Las **pruebas funcionales** verifican si una aplicación de software se comporta según los requisitos especificados.

Las **pruebas no funcionales** evalúan el rendimiento del software en diversas condiciones, como carga, estrés o en diferentes entornos.

Las siguientes listas describen los tipos de pruebas más comunes dentro de cada categoría.



Pruebas de Software

Tipos de pruebas de software

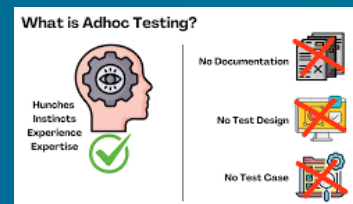
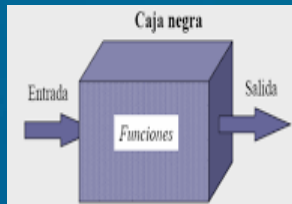
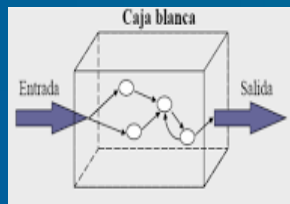
Tipos de pruebas funcionales

Pruebas de caja blanca: implican pruebas basadas en el conocimiento de la estructura interna, la lógica y las funciones del software que se está probando.

Pruebas de caja negra: un evaluador no tiene ninguna información sobre el funcionamiento interno del sistema de software.

Pruebas ad hoc: los evaluadores intentan romper o encontrar errores en una aplicación sin seguir pruebas o documentación predefinidas.

Pruebas de API (interfaz de programación de aplicaciones): verifican que las interfaces entre los componentes de software funcionen de forma correcta y fiable. Las pruebas de API son una parte esencial de la gestión de API, el software y los procesos que respaldan el ciclo de vida de una API.



Pruebas de Software

Tipos de pruebas de software

Tipos de pruebas funcionales

Pruebas exploratorias: ayudan a los evaluadores de software a descubrir escenarios y situaciones difíciles de predecir que pueden provocar errores en el software.

Pruebas de regresión: comprueban si las nuevas características rompen o degradan la funcionalidad existente. Garantizan que los cambios recientes no hayan introducido nuevos defectos.

Pruebas de cordura: evalúan si las funcionalidades específicas funcionan según lo esperado. Los evaluadores pueden utilizarlas para verificar menús, funciones y comandos a nivel superficial cuando no hay tiempo para realizar una prueba de regresión completa.



Pruebas de Software

Tipos de pruebas de software

Tipos de pruebas funcionales

Pruebas de humo: son un proceso preliminar de prueba de software que comprueba si las funciones básicas de una aplicación funcionan correctamente. Ayudan a garantizar que la compilación sea lo suficientemente estable como para realizar más pruebas.

Pruebas de aceptación del usuario (UAT): son un tipo específico de pruebas de aceptación realizadas por los usuarios finales para confirmar que el sistema satisface sus necesidades y funciona en escenarios del mundo real.



Pruebas de Software

Tipos de pruebas de software

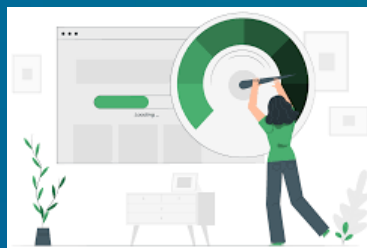
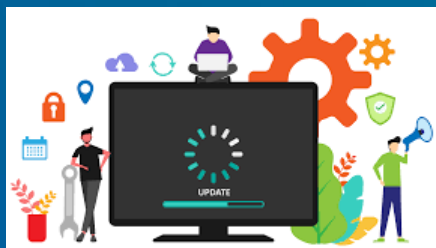
Tipos de pruebas no funcionales

Pruebas de recuperación: verifican cómo el software responde y se recupera de los fallos, garantizando que los datos y los procesos se restauren correctamente.

Pruebas de rendimiento: se refieren a cómo se ejecuta el software bajo diferentes cargas de trabajo.

Pruebas de carga: tipo de prueba de rendimiento, evalúan el rendimiento en condiciones reales de equilibrio de carga.

Pruebas de estrés: examinan la cantidad de tensión que el sistema puede soportar antes de que falle.



Pruebas de Software

Tipos de pruebas de software

Tipos de pruebas no funcionales

Pruebas de seguridad: validan si el software está abierto a hackers u otros tipos de vulnerabilidades maliciosas.

Pruebas de usabilidad: validan lo bien que un cliente puede utilizar la interfaz de usuario de un sistema para completar una tarea de forma eficaz e intuitiva.

Pruebas de compatibilidad: comprueban si una aplicación de software funciona según lo previsto en varios dispositivos, sistemas operativos, navegadores y entornos de red.

Pruebas de Software

Buenas prácticas de pruebas de software

Las pruebas de software eficaces comienzan con un plan de pruebas sólido que describe el alcance, El enfoque y los recursos necesarios para validar el software a lo largo de su ciclo de vida.



Para escalar y adaptarse a entornos complejos, los equipos de pruebas confían en un marco de pruebas robusto, una estructura que admite la automatización, se integra con los pipelines de CI/CD y permite la validación continua en todas las plataformas y entornos. Estos marcos admiten todas las fases de las pruebas, incluido el diseño, la ejecución y el análisis de los resultados, lo que ayuda a los equipos a detectar problemas antes, reducir el riesgo y mejorar el tiempo de lanzamiento. Las revisiones de código también desempeñan un papel fundamental en el control de calidad, ya que permiten a los equipos detectar defectos antes y aplicar las normas de codificación incluso antes de que comiencen las pruebas.

Pruebas de Software

Buenas prácticas de pruebas de software

Las pruebas pueden llevar mucho tiempo. Las pruebas manuales o ad hoc pueden ser suficientes para compilaciones pequeñas. Sin embargo, en el caso de sistemas más grandes, se suelen utilizar herramientas para automatizar las tareas. Las pruebas automatizadas ayudan a los equipos a implementar diferentes escenarios, probar diferenciadores (como trasladar componentes a un entorno en la nube) y obtener rápidamente feedback sobre lo que funciona y lo que no.

Un enfoque de pruebas sólido abarca la interfaz de programación de aplicaciones (API), la interfaz de usuario y los niveles del sistema. Cuantas más pruebas se automaticen y se ejecuten en una fase temprana, mejor. Algunos equipos crean herramientas de automatización de pruebas internas. Sin embargo, las soluciones de los proveedores ofrecen características que pueden agilizar tareas clave de gestión de pruebas, tales como:

Pruebas de Software

Buenas prácticas de pruebas de software

Pruebas continuas: los equipos de proyecto prueban cada compilación a medida que está disponible. Este tipo de pruebas de software se basa en la automatización de pruebas que se integra con el proceso de implementación, a menudo como parte de la implementación continua. Permite validar el software en entornos de prueba realistas en una fase más temprana del proceso de desarrollo, lo que mejora el diseño y reduce los riesgos.

Gestión de la configuración: las organizaciones mantienen de forma centralizada los activos de prueba y realizan un seguimiento de las compilaciones de software que se están probando. Los equipos obtienen acceso a activos como código, requisitos, documentos de diseño, modelos, scripts de prueba y resultados de pruebas. Los sistemas sólidos incluyen la autenticación de usuarios y los registros de auditoría para ayudar a los equipos a cumplir con los requisitos de cumplimiento con un esfuerzo administrativo mínimo.

Pruebas de Software

Buenas prácticas de pruebas de software

Virtualización de servicios: es posible que los entornos de prueba no estén disponibles, especialmente al principio del desarrollo del código. La virtualización de servicios simula los servicios y sistemas que faltan o que aún no se han completado, lo que permite a los equipos reducir las dependencias y realizar las pruebas antes. Pueden reutilizar, implementar y cambiar una configuración para probar diferentes escenarios sin tener que modificar el entorno original.

Seguimiento de defectos o errores: la monitorización de defectos y el seguimiento de errores son cruciales tanto para los equipos de pruebas como para los de desarrollo, ya que les permiten medir y mejorar la calidad del software. Las herramientas automatizadas permiten a los equipos hacer un seguimiento de los defectos, medir su alcance e impacto y descubrir problemas relacionados.

Pruebas de Software

Buenas prácticas de pruebas de software

Métricas e informes: los informes y los análisis permiten a los miembros del equipo compartir el estado, los objetivos y resultados de las pruebas. Las herramientas de prueba avanzadas integran las métricas del proyecto y presentan los resultados en un panel de control. Los equipos ven rápidamente el estado general de un proyecto y pueden monitorizar las relaciones entre las pruebas, el desarrollo y otros elementos del proyecto.

Pruebas de Software

Buenas prácticas de pruebas de software

El futuro de las pruebas de software

A medida que el ritmo del desarrollo de software se acelera y los sistemas se vuelven cada vez más complejos, las pruebas de software continúan evolucionando al mismo tiempo. A continuación se presentan algunas tendencias clave que están dando forma al futuro de las pruebas.

Pruebas low-code y no-code

A medida que las plataformas low-code y no-code siguen ganando popularidad, están surgiendo nuevas herramientas de pruebas de software que se adaptan a usuarios no técnicos. Estas herramientas simplifican los procesos de prueba, lo que permite a los usuarios empresariales crear y ejecutar fácilmente pruebas en las aplicaciones que crean. Esta capacidad acelera el tiempo de comercialización sin necesidad de conocimientos técnicos.

Pruebas de Software

Buenas prácticas de pruebas de software

El futuro de las pruebas de software

IoT y pruebas de edge

La rápida expansión de los dispositivos del Internet de las cosas (IoT) presenta desafíos únicos en términos de pruebas de conectividad, seguridad y rendimiento en condiciones reales. A medida que más dispositivos dependen del edge computing, las herramientas de prueba deben simular diversos entornos para garantizar que el software pueda funcionar de forma fiable en diversas condiciones de red.

Pruebas de 5G y latencia ultrabaja

Desde la llegada del 5G, las aplicaciones que requieren una latencia ultrabaja, como los vehículos autónomos y la asistencia sanitaria a distancia, necesitan pruebas especializadas. Validar el rendimiento en condiciones de alta velocidad y baja latencia se ha convertido en crucial para las aplicaciones móviles y basadas en el edge.

Pruebas de Software

Buenas prácticas de pruebas de software

Sistemas predictivos y de autorreparación impulsados por IA

Los sistemas de autorreparación impulsados por IA detectan y corrigen automáticamente problemas menores, lo que reduce el tiempo de inactividad. Las pruebas predictivas, impulsadas por el machine learning (ML), permiten a los equipos anticipar posibles fallos y abordarlos antes de que interrumpan la producción, lo que a su vez hace que el software sea más resiliente y fiable.

La IA generativa en pruebas

La IA generativa se ha convertido en una poderosa herramienta en las pruebas de software. Mediante el uso de técnicas avanzadas de machine learning, los modelos de IA generativa pueden crear casos de prueba dinámicos basados en el comportamiento del software. Estos modelos generan automáticamente nuevos escenarios que los evaluadores humanos podrían pasar por alto, lo que en última instancia mejora la cobertura de las pruebas y la fiabilidad del software.